



USJ

**ESCUELA DE
ARQUITECTURA
Y TECNOLOGÍA**

INGENIERÍA
INFORMÁTICA
+ DISEÑO Y
DESARROLLO DE
VIDEOJUEGOS

HTTP group project

LET'S BUILD THE WEB FROM SCRATCH



USJ

ESCUELA DE
ARQUITECTURA
Y TECNOLOGÍA

INGENIERÍA
INFORMÁTICA
+ DISEÑO Y
DESARROLLO DE
VIDEOJUEGOS

What if your favourite framework **disappeared**
tomorrow? 🤔



USJ

**ESCUELA DE
ARQUITECTURA
Y TECNOLOGÍA**

INGENIERÍA
INFORMÁTICA
+ DISEÑO Y
DESARROLLO DE
VIDEOJUEGOS

You've got two months to find out 🤔

What is this project about?

You will implement a **simplified HTTP/1.1 client and server** from scratch, using only **TCP sockets**.

No Express. No Axios. No Spring. No shortcuts.

- Your client talks to real servers (`example.com` , any website)
- Your server responds to real clients (`curl` , browsers)
- **Interoperability** is a key evaluation criterion

Why should you care?

- Every web framework you'll ever use sits **on top of HTTP**
- Understanding the protocol means you can **debug anything**
- Security issues live at this layer (headers, TLS, auth)
- Protocol design patterns appear everywhere: REST, gRPC, WebSockets

This is not about memorising RFCs. It's about **building intuition** for how networked systems actually work.

The constraint

You can use **any programming language** you want.

The only rule: no libraries above the transport layer.

Language	Allowed (TCP sockets) ✓	Not allowed (HTTP libs) ✗
Node.js	<code>net</code>	Express, Axios
Java	<code>java.net.Socket</code>	Spring, OkHttp
Python	<code>socket</code>	requests, Flask

You are building the HTTP library yourselves.



Two deliverables: **client** + **server** 

HTTP Client

A command line program that can:

- Send requests to **any URL** with any HTTP verb
- Set custom **headers** and **body**
- Display the full response (status, headers, body)
- Send **multiple requests** without restarting

Think of it as your own mini- `curl`.



USJ

ESCUELA DE
ARQUITECTURA
Y TECNOLOGÍA

INGENIERÍA
INFORMÁTICA
+ DISEÑO Y
DESARROLLO DE
VIDEOJUEGOS



HTTP Server with RESTful API

A server that exposes:

- A **static file** endpoint (e.g., `/index.html`)
- A full **REST CRUD** on a resource of your choice:
 - `GET /cats`, `GET /cats/:id`, `POST /cats`, `PUT /cats/:id`, `DELETE /cats/:id`
- Proper **status codes**: `200`, `201`, `204`, `400`, `404`, `405`
- **JSON** request and response bodies
- **Concurrent** request handling



Optional features

Pick optional features to raise your maximum grade:

Low difficulty

- API key authentication
- Logging
- Cookies
- Cloud deployment
- Middleware chain
- Advanced CRUD

Medium/High difficulty

- Login flow with tokens
- Multimedia (MIME)
- TLS (basic or advanced)
- Automated testing
- HTTP/1.1 compliance
- Client GUI



Groups and grading

- Groups of **4 to 6** students (5 recommended)
- You choose your **maximum grade** based on features implemented

Feature type	4 students	5 students	6 students
Mandatory	+7 pts.	+6 pts.	+5 pts.
Low optional	+0.8 pts.	+0.6 pts.	+0.5 pts.
Medium optional	+1.5 pts.	+1.2 pts.	+1 pts.
High optional	+2.5 pts.	+2 pts.	+1.8 pts.

Finish mandatory features **first**, then add optionals.

Evaluation breakdown

Area	Weight
Technical report	30%
Live demo (10 min)	30%
Code quality	40%

Code quality includes **protocol correctness** (proper parsing, `\r\n`, Content-Length, status codes...).

Individual grades are adjusted via **anonymous peer review** (x0.8 to x1.2).

Delivery

- 📦 **Report + repo**: due **May 13, 2026**. Report via PDU, must link to your repo
- 🎤 **Live demos: May 18 and 20, 2026** (10 min per group)
- 👤 **Repository**: invite me as collaborator (`@pitazzo` on GitHub)
- 🤖 **AI tools**: allowed and encouraged, but you **must document** usage and **explain every line** during the demo



Suggested approach



Step by step

1. Build a **client library** that wraps TCP sockets into clean HTTP calls
2. Build a **server library** that parses incoming requests and routes them
3. Wire the client library into an **interactive CLI**
4. Serve a **static HTML file** from disk
5. Implement the **REST API** with in-memory persistence
6. Handle **error cases** (400 , 404 , 405)

Starter code examples are available in the [project repository](#) (JS, Java, Python).



USJ

**ESCUELA DE
ARQUITECTURA
Y TECNOLOGÍA**

INGENIERÍA
INFORMÁTICA
+ DISEÑO Y
DESARROLLO DE
VIDEOJUEGOS

Go build the web! 
Statement in [the repo](#).